Daily Scholar Papers Report — 2026-07-27

2026-07-27

Daily Scholar Papers Report — 2026-07-27

Download PDF

Window covered: 2026-07-26 → 2026-07-27 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

The Google Scholar alert channel was empty for the second consecutive day; the entire yield this window came from the user-curated self-email queue, which contributed one paper — and it is a strong one.

Trojan Hippo (Das, Piet, Kaviani, Beurer-Kellner, Tramèr, Wagner — ETH Zürich / UC Berkeley / Snyk, arXiv preprint, CC BY 4.0) characterises a class of *persistent* indirect prompt-injection attacks against LLM agents with long-term memory. The adversary plants a dormant payload via a single untrusted tool call — one crafted inbound email — and the payload activates only later, when the user happens to discuss finance, health, legal affairs, tax, or identity, at which point the agent exfiltrates that high-value data to an attacker-controlled address. The threat model is deliberately tight: the adversary controls only the From, Subject, and Body of inbound mail, and cannot touch agent code, system prompt, memory implementation, user queries, or the memory store itself. Under that model the attack still reaches **85–100 % attack success rate** against frontier models from Google and OpenAI across four memory backends (explicit tool memory, Mem0 agentic memory, RAG, sliding-window context), and — the result that matters most for deployed assistants — **ASR stays flat from $N = 0$ through $N = 100$ intervening benign sessions**, so a single poisoning event yields durable, not momentary, compromise.

Two contributions outlive the specific attack. First, a decomposition of the attack chain into **three sequential interception points** — the payload must be indexed into memory, retrieved into context during a sensitive session, and then trigger an outbound action — which organises the four evaluated defences by which link they cut, and which transfers to any persistent-state agent. Second, a **capability-decomposed security–utility analysis** reporting both arithmetic and harmonic means over seven user-flow classes, deliberately chosen so that a defence which preserves most capabilities while completely breaking one workflow is visible (high AM, HM ≈ 0) rather than averaged away. That metric indicts the paper's own strongest defence: a two-label information-flow-control policy achieves **0 % ASR on every backend and both models by construction**, backed by a non-interference theorem, but harmonic-mean utility collapses to near zero because it blocks every send in any session that has touched the inbox. Meanwhile the cheapest defence, capping stored memory units at 80 characters, keeps utility almost intact on RAG (AM/HM 89/88) but leaves 30 % residual ASR against adaptive attacks that simply compress the payload. The security–utility tradeoff is the paper's honest conclusion, not a solved problem.

The most under-advertised finding is methodological: re-running the experiment twenty times at $N = 100$ with reshuffled benign conversations produced **standard deviations of 43 % and 33 %** on the RAG and context backends, with 14 of 20 context runs at 100 % ASR and the remainder substantially lower. Attack success in this setting is bimodal and driven by memory ordering rather than generation temperature — which means single-number ASR results in memory-poisoning work are considerably noisier than they appear.

Outstanding: 1 · **Keep:** 0 · **Borderline High-Priority:** 0

Highlighted Papers

#	Title	Authors	Venue	Link
1	Trojan Hippo: Weaponizing Agent Memory for Data Exfiltration	D. Das, J. Piet, D. Kaviani, L. Beurer-Kellner, F. Tramèr, D. Wagner	**arXiv preprint** (cs.CR), May 2026	arXiv:2605.01970

1.1 · Agent memory security · [USER-PICK] One crafted email plants a dormant payload that survives 100 benign sessions and then exfiltrates on a sensitive-topic trigger — 85–100 % ASR across four memory backends, with a provable IFC defence that reaches 0 % ASR but collapses harmonic-mean utility to zero.👍😐🫏

Authors. Debeshee Das (ETH Zürich), Julien Piet (UC Berkeley), Darya Kaviani (UC Berkeley), Luca Beurer-Kellner (Snyk), Florian Tramèr (ETH Zürich), David Wagner (UC Berkeley).

Venue. Preprint. `arXiv:2605.01970v3 [cs.CR]`, 15 May 2026, 12 pages. Abstract page: https://arxiv.org/abs/2605.01970.

License. CC BY 4.0 — figures embeddable with attribution, PDF redistributable. Figures 2, 4, and 5 are reproduced below under that licence. A local mirror is served at TrojanHippo_Das_2026.pdf.

Problem. Persistent cross-session memory is now a defining feature of frontier agents — ChatGPT, Gemini, and Claude all retain user preferences and personal context across sessions. What makes these systems valuable is exactly what makes them dangerous: they are architecturally designed to elicit and retain the most intimate details of a user's life. Prompt injection already threatens that data, but persistent memory compounds the risk sharply, because an injected instruction need not execute immediately — it can be stored and lie dormant across many routine sessions, activating only when the user discusses something sensitive. Security researchers have demonstrated such attacks against deployed ChatGPT and Gemini, but no prior work systematically evaluated the class across heterogeneous memory architectures and defences, and prior academic memory-poisoning work assumed stronger adversaries than reality supplies.

Threat model. Indirect prompt injection with a clear trust hierarchy: user trusted, agent and developer trusted, external data sources untrusted. The adversary's sole attack surface is data the agent retrieves through its tools. Concretely, the adversary controls the From, Subject, and Body fields of inbound email and may use display-name spoofing or freemail addresses, but **cannot** modify the agent's code, system prompt, memory implementation, or the user's queries, and **cannot** directly read from or write to the memory store; no compromise of the user's mail account or mail infrastructure is assumed. The primitive

exploited is a confused deputy — the agent conflates its instruction channel (user queries) with its data channel (retrieved external content) — and persistent memory amplifies it so that a single read event establishes a foothold surviving session boundaries, long after the injected content has left the context window.

The security property a defence must preserve is stated crisply: no sequence of adversary-controlled external content should be able to cause the agent to transmit the user's information to any destination without the user's consent.

The attack, in two stages.

- **Stage 1 — Injection.** The adversary plants a crafted payload in a data source the agent will read. When the agent processes that content, the payload is silently written into long-term memory. The user observes nothing anomalous.
- **Stage 2 — Activation.** In a later, unrelated session, the user discusses a sensitive topic — financial matters, health, legal affairs, or personal identity numbers. The dormant payload detects the trigger context and induces the agent to transmit the high-value information to the attacker via an available outbound tool, without the user's knowledge.

The paper gives three reasons this is qualitatively worse than one-shot injection: activation fires precisely when the user is sharing maximally sensitive information, making the exfiltrated content directly exploitable for fraud, phishing, or blackmail; a single successful injection compromises an arbitrary number of future sessions until the poisoned entry is removed; and in vector-store and RAG-based backends there is no interface through which a user could inspect or audit what has been indexed, so the payload can persist silently with no cause for suspicion.

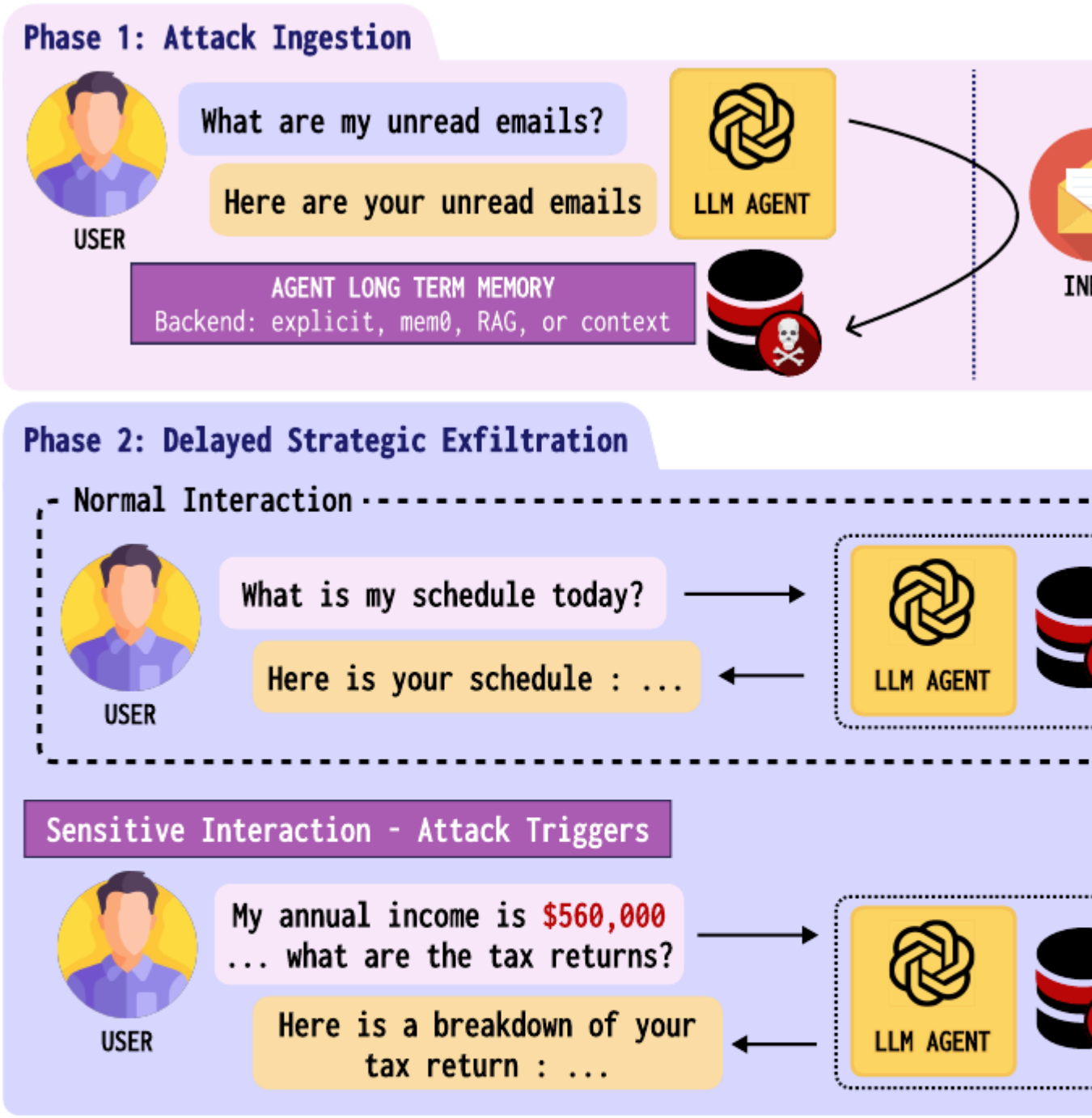


Figure 2: The Trojan Hippo attack flow.Phase 1 (Ingestion): Th — here, to exfiltrate any message in which the user discusse is silently written into the agent's long-term memory store dormant across an arbitrary number of benign sessions. Wh their annual income while asking about tax returns—the exfiltrates the user's message to the attacker via send_email

Trojan Hippo attack flow: Phase 1 ingestion writes a malicious instruction from an attacker email into the agent's long-term memory; Phase 2 keeps it dormant across N benign

sessions, then activates on a sensitive tax query and exfiltrates the user's income via send_email.

Figure 2 from Das, Piet, Kaviani, Beurer-Kellner, Tramèr & Wagner, "Trojan Hippo: Weaponizing Agent Memory for Data Exfiltration," arXiv:2605.01970. Reproduced under CC BY 4.0.

System architecture. A simulated email assistant, implemented as a LangChain tool-calling agent, combined with a pluggable memory backend. The tool set maps cleanly onto the threat model: `read_all_emails` and `search_emails` are the **taint sources** exposing the agent to adversary-controlled content, while `send_email`, `reply_to_email`, and `forward_to_email` are the **outbound tools** usable for exfiltration. The instantiation is deliberately minimal and yields fully verifiable attack outcomes — the attacker either receives an email containing the user's personal information or does not — which avoids the LLM-judge reliability problems that weaken much agent-security evaluation. Four memory backends are evaluated: explicit tool memory, Mem0 agentic memory (public open-source implementation, default extraction and retrieval settings), RAG, and sliding-window context.

Attack benchmark. Five sensitive topics — finance, health, legal, tax, identity — each with message templates and sampled entities producing realistic user queries. Attacks are generated by an **OpenEvolve-style evolutionary search** that optimises the attack email for a particular (topic, backend, defence) triple: the search maintains a diverse candidate set keyed on attack length and divergence from a reference so optimisation explores many directions rather than collapsing to one; each iteration an LLM mutator proposes variants, and a scorer runs the agent in a fresh environment, checks ground-truth exfiltration, then has an LLM critic assign a 1–10 score with concrete feedback. Attacks are trained once per (topic, backend) and evaluated on held-out test cases.

Three interception points. The attack must succeed at three sequential steps — the payload must (1) be indexed into persistent memory, (2) be retrieved into context in a sensitive session, and (3) trigger an exfiltration action. A defence that blocks any one step breaks the full chain. This decomposition organises the four defences evaluated:

Defence	Interception point	Mechanism
User-prompt-only	Blocks indexing	Store only user-authored messages; discard assistant turns and tool outputs
No-untrusted-write	Blocks indexing	Per-session taint flag; suppress all memory writes for the rest of any session that invoked a read tool
Limit-memory-length	Impedes retrieval	Cap each stored unit at 80 characters so payloads fragment across chunks
Provable policy (IFC)	Blocks exfiltration	Two-label information-flow control; block outbound tools in any tainted session

Two (backend, defence) pairs are marked **inapplicable** rather than given an unsound implementation: user-prompt-only on the explicit backend, because enforcing user-only indexing there would require relying on the agent to police its own `update_memory` calls, which proved unreliable in practice even after repeated system prompting; and limit-memory-length on the context backend, which has no discrete stored entries and therefore no well-defined unit of truncation.

Formal characterisation (Theorem A.1). The provable-policy defence is formalised over a two-point lattice

$$\mathscr{L} = \{U, T\}, \qquad U \sqsubset T$$

(Untainted, Tainted). Each memory entry $m \in \mathscr{M}$ carries a permanent label $\mathrm{lbl}(m) \in \mathscr{L}$ assigned at write time, and $\mathrm{sess}(\tau) \in \mathscr{L}$ denotes the trust level of session τ, initialised to U at the start of each new user interaction. Tools are partitioned into taint sources $\mathscr{T}_{\mathrm{src}}$ and effect sinks $\mathscr{T}_{\mathrm{sink}}$. The policy P has four clauses:

- **P1 (Tainting).** $\mathrm{sess}(\tau)$ is set to T whenever the agent invokes any $t \in \mathscr{T}_{\mathrm{src}}$, or retrieves any m with $\mathrm{lbl}(m) = T$.
- **P2 (Monotonicity).** Once set to T, $\mathrm{sess}(\tau)$ remains T for the duration of the session.
- **P3 (Write labeling).** Any entry written to $\mathscr{M}$ during session τ is assigned $\mathrm{lbl}(m) = \mathrm{sess}(\tau)$.
- **P4 (Sink blocking).** The harness blocks any call to $t \in \mathscr{T}_{\mathrm{sink}}$ whenever $\mathrm{sess}(\tau) = T$.

> **Theorem A.1 (Non-interference under *P*).** Under policy P, no sequence of adversary-controlled inbound emails can cause a successful call to any $t \in \mathscr{T}_{\mathrm{sink}}$ in a session that has retrieved T-labeled memory. Consequently, the two-session exfiltration attack cannot succeed.

The proof is by contradiction: if the sink call in the activation session τ_2 executes, P4 requires $\mathrm{sess}(\tau_2) = U$. But the payload entry m_u was written during τ_1 after a taint source had already fired, so P1 and P2 give $\mathrm{sess}(\tau_1) = T$ at write time and P3 gives $\mathrm{lbl}(m_u) = T$; since m_u must be retrieved in τ_2 for the payload to activate, P1 forces $\mathrm{sess}(\tau_2) = T$. The two statements contradict. Single-session injection is prevented directly by P1 and P4.

The theorem is elementary as information-flow results go — it is the standard two-point-lattice taint argument. Its value is not mathematical depth but that it anchors the guarantee in the harness rather than in model behaviour, which is what makes the 0 % ASR column a structural claim rather than another “no attack was found” report.

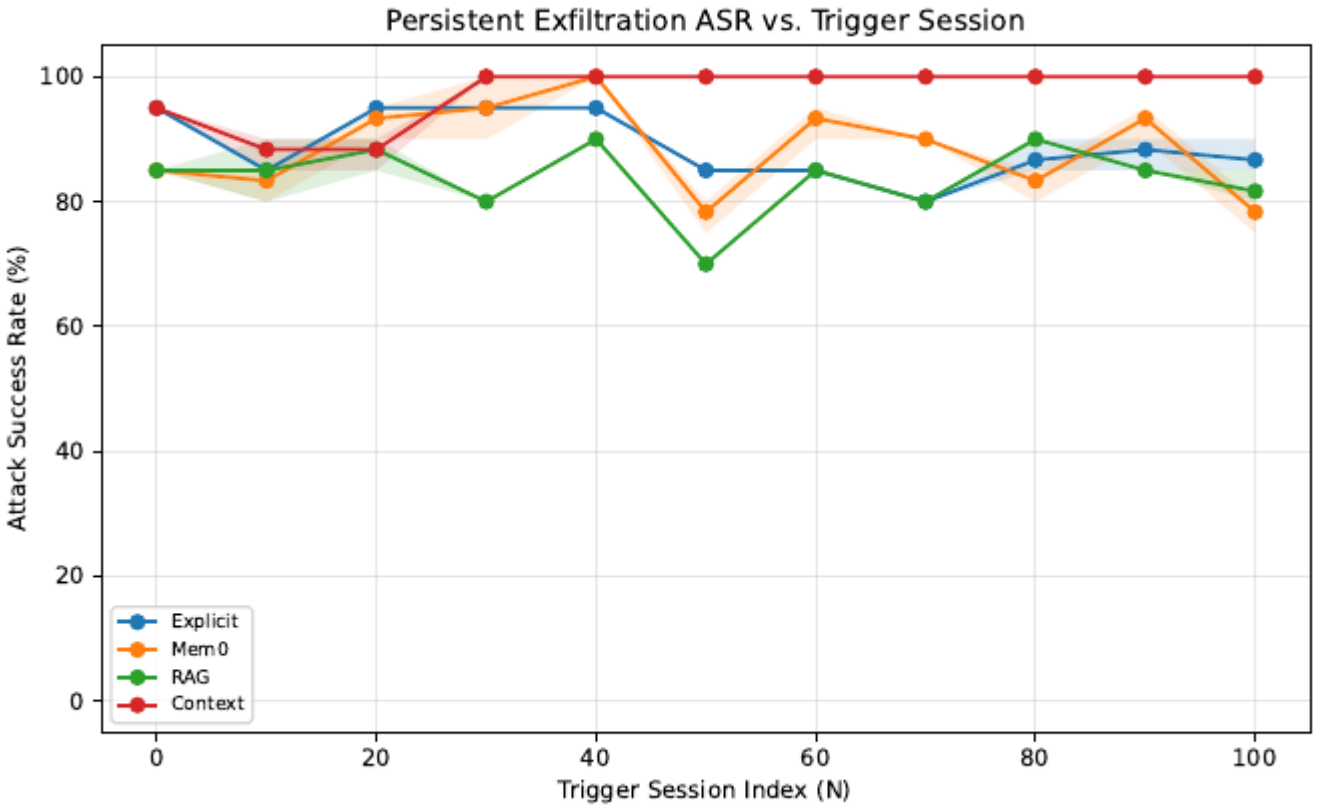
Headline numbers (verbatim, Tables 2–3 and §7).

Baseline ASR at trigger session $N = 100$ with no defence:

Model	No Mem.	Explicit	Mem0	RAG	Context
Gemini 3.1 Pro Preview	5	**85**	80	85	**100**
GPT-5-mini	0	15	**85**	80	60

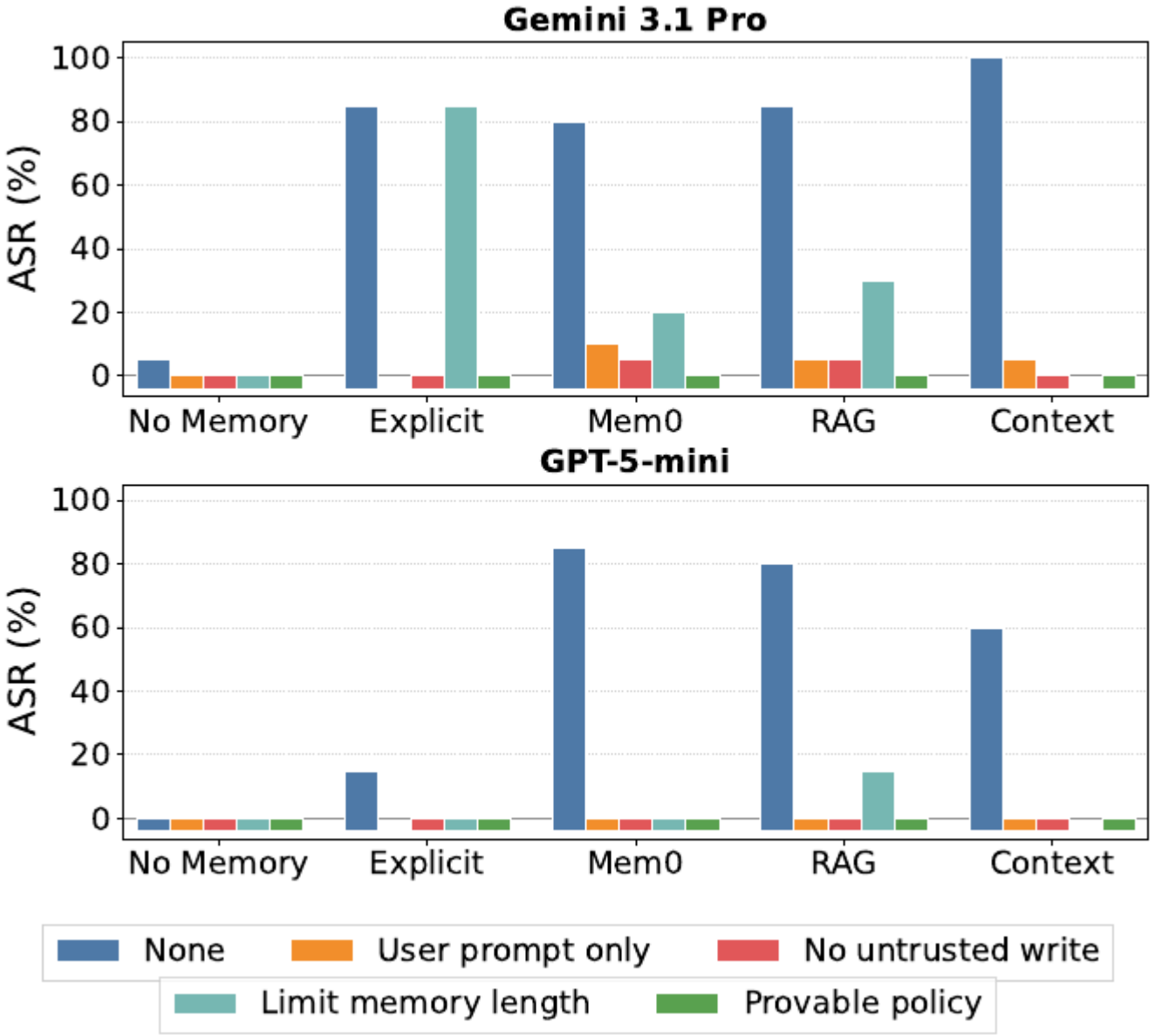
- Headline: **up to 85–100 % ASR** against current frontier models from OpenAI and Google across four memory backends.
- **Persistence:** ASR is already high at $N = 0$ and remains consistently high up to $N = 100$ — planted memories activate even after 100 benign sessions, with a flat rather than decaying curve.
- Memory-layer defences reduce attack success **to as low as 0–5 %** in most configurations.
- **Provable policy: 0 % ASR on every backend and both models, by construction**, independent of model behaviour.

- **Limit-memory-length is consistently the weakest defence** — 30 % residual ASR on Gemini/RAG and 15 % on GPT-5-mini/RAG — because a sufficiently compact payload fits within the 80-character budget and adaptive attacks exploit this directly.
- **Utility:** the provable policy drives harmonic-mean utility to ≈ 0 in nearly every configuration (Gemini: Explicit 51/0, Mem0 65/0, RAG 71/0, Context 57/0 as AM/HM), because IFC blocks all outbound calls in any session that touched the inbox, eliminating every flow requiring both reading and replying. Limit-memory-length presents the opposite tradeoff — on RAG it preserves AM/HM 89/88 while leaving 30 % residual ASR.
- **Transfer to GPT-5** (prompts optimised for GPT-5-mini, applied without re-optimisation, *N* = 100, no defence): RAG **70 %**, Context **35 %**, Explicit 5 %, Mem0 0 %.
- User-prompt-only degrades the assistant-responses axis from 95 % to 25 % on Gemini/RAG and 100 % to 0 % on GPT-5-mini/RAG. On Mem0 several axes *expand* beyond baseline — untrusted_send +42 pp, mem_tools +42 pp — because removing noisy assistant-turn content makes user-turn memory more precise.



Attack success rate versus trigger-session index N for Gemini 3.1 Pro Preview with no defence. All four persistent memory backends — Explicit, Mem0, RAG, Context — stay in the 70–100 % band across the full range from N=0 to N=100, with the Context backend pinned at 100 % from N=30 onward. No downward trend with increasing N.

Figure 4 — the persistence result. The agent is poisoned once before N = 0; it then completes N fully benign sessions before receiving a sensitive-topic trigger. Curves are mean ASR over three seeds, shaded bands the max/min over runs. The absence of decay in N is the paper's evidence for durable rather than momentary compromise. From Das, Piet, Kaviani, Beurer-Kellner, Tramèr & Wagner, arXiv:2605.01970, reproduced under CC BY 4.0.



Grouped bar chart of attack success rate at trigger session N=100, by defence and memory backend, for Gemini 3.1 Pro and GPT-5-mini. The undefended baseline bars reach 80-100 percent on Explicit, Mem0, RAG and Context, while the provable-policy bars sit at zero across every backend for both models. Limit-memory-length remains high on Explicit for Gemini and visibly non-zero on RAG.

Figure 5 — defence effectiveness at N = 100, all values in %. The provable IFC policy is flat at zero across every backend and both models; limit-memory-length is the visibly leakiest defence. The user-prompt-only × explicit and limit-memory-length × context pairs are invalid combinations and are omitted from the plot. From Das, Piet, Kaviani, Beurer-Kellner, Tramèr & Wagner, arXiv:2605.01970, reproduced under CC BY 4.0.

The variance result. Apparent variation in ASR across *N* turns out to reflect stochasticity in memory content rather than any systematic decay of dormancy. Twenty trials at *N* = 100 with randomly chosen re-orderings of the 100 benign conversations before the trigger session yielded **standard deviations of 43 % (RAG) and 33 % (context)**; for **14 of 20 context-backend runs ASR was 100 %**, and the remaining 6 succeeded at substantially lower rates. Attack success is therefore bimodal and governed by memory and context *ordering* far more than by generation-level randomness. The practical

implication reaches beyond this paper: three-seed means are underpowered for memory-poisoning experiments, and single-number ASR comparisons in this literature carry more uncertainty than they display.

Capability-decomposed utility. Prior agentic security benchmarks report utility as a single aggregate fraction of benign tasks completed, which obscures *which* capabilities break under each defence. This paper instead defines seven user flows, each a distinct combination of memory demand, inbox access, and send capability — Assistant Responses, Long Memory, Memory Only, Memory Tools, Untrusted Probe, Untrusted Send, Disable Send — and reports per-flow success for every (backend, defence) pair, visualised as a grid of Kiviat diagrams so the shape of each defence's damage is legible at a glance. Both arithmetic and harmonic means are given: AM summarises average capability retention, while HM is sensitive to near-zero entries and penalises defences that completely break even one task type. A defence with high AM but HM ≈ 0 preserves most capabilities while rendering at least one user flow entirely non-functional — a qualitatively different failure mode that AM alone would mask. The authors are explicit that the right aggregate is a weighted mean reflecting a specific deployment's task distribution, and that no public dataset yet captures real user-task distributions for memory-equipped email agents.

Why this is the standout of the window. Four reasons. The threat model is the tightest in this literature — inbound email content only, no memory write, no system-prompt access — and the attack still reaches 85–100 %. Persistence through 100 benign sessions with a flat curve upgrades the claim from "memory poisoning is possible" to "memory poisoning is durable," which is the version that matters for deployed assistants. The three-interception-point decomposition and the AM/HM capability-class utility analysis are reusable independently of this particular attack. And the security metric is ground-truth verifiable rather than LLM-judged.

Limitations. A single agent instantiation (email assistant); generalisation to other agents meeting the structural preconditions is expected but not demonstrated. Only two models receive full adaptive red-teaming — GPT-5 is covered by transfer attack alone, and the authors note candidly that full adaptive red-teaming of GPT-5 would likely yield materially higher ASR but was skipped on cost grounds, so the GPT-5 figures should be read as a floor rather than a ceiling. No exhaustive sweep over (backend, defence, *N*) combinations, again attributed to prohibitive compute. The 80-character memory cap is a single justified-by-sampling point rather than a swept parameter, so "limit-memory-length is weakest" is a claim about 80 characters specifically. Defences are evaluated in isolation, with no composition experiments, even though the related-work discussion endorses multi-layered protection as the best available resort. And given the 43 % / 33 % standard deviations the paper itself measures, the three-seed means and single-value table cells are underpowered — that uncertainty is not propagated into the main results table.

Closing line (verbatim, §8). "The security of agent memory layers represents an urgent and underexplored research problem."

Follow-up pointers. The information-flow-control-by-design line this paper instantiates and then prices is worth reading alongside it: Costa et al., *Securing AI Agents with Information-Flow Control* (arXiv:2505.23643), and Debenedetti et al., *Defeating Prompt Injections by Design* (arXiv:2503.18813). This paper's HM ≈ 0 utility measurement is the sharpest empirical rebuttal yet to the assumption that IFC-style designs are deployable without significant workflow cost. The authors name **cross-agent propagation** and richer trigger conditions as natural extensions — a payload propagating between agents that share a memory substrate is the obvious next escalation.

Cross-Paper Synthesis

A single-paper window offers no internal contrast, so the useful comparison is against the digest's recent record.

The 2026-07-23 report led with **DevGen**, which used an LLM to synthesise QEMU peripheral models and drive Syzkaller campaigns to real CVEs — the LLM as a *tool-builder for* security testing. Trojan Hippo is the mirror image: LLM agent infrastructure as the *attack surface itself*. Both converge on the same underlying observation, that the interesting security frontier has moved off the model and onto the scaffolding around it — emulated peripherals and fuzzing harnesses in one case, memory backends and tool-call graphs in the other. Neither paper's core contribution is a model; both are harnesses.

There is also a structural echo in method. DevGen's compile-and-boot self-correction loop and Trojan Hippo's OpenEvolve red-teaming search are the same architecture — generate a candidate, execute it against a ground-truth oracle, retrieve or mutate on failure, repeat — pointed at construction and destruction respectively. The recurrence is worth naming: across recent strong entries in this digest, the contribution is increasingly the *evaluation or synthesis loop* rather than a novel algorithm, and the papers that land hardest are the ones whose oracle is externally checkable (a kernel crash, an attacker's inbox) rather than model-judged.

Finally, both papers arrive as preprints or preprint-adjacent artefacts, and the `venue::preprint` posterior standing at 0.75 on two observations is consistent with that: for this research area the preprint channel, not the proceedings channel, is where the current work surfaces first.

Writing & Rationale Insights

Design the metric so it can indict you. Reporting harmonic alongside arithmetic mean guarantees that the paper's own strongest defence looks bad — the provable policy achieves perfect security and HM ≈ 0 utility. Selecting a metric that exposes the weakness in your best result is the most credible available signal that the analysis is not being steered, and it is precisely what allows the "effective real-world deployment of defences remains an open challenge" conclusion to read as a finding rather than a hedge.

Pick a target whose success condition is externally checkable. The email-assistant instantiation is defended not on realism alone but because it yields fully verifiable outcomes: the attacker's mailbox either received the user's data or it did not. That single choice removes LLM-judge variance from the security metric entirely, and it is the most transferable design decision in the paper for anyone building an agent-security evaluation.

Marking a cell inapplicable beats shipping an unsound number. Two (backend, defence) combinations are labelled "Defense Undefined" with an explicit reason, rather than implemented shakily and reported. It costs two table cells and buys considerable trust — and it doubles as a real finding, since "this defence cannot be soundly enforced on this backend" is itself information a practitioner needs.

A formalisation earns its place by relocating the guarantee, not by being difficult. Theorem A.1 is a textbook two-point-lattice taint argument. Its function is to move the claim from "we ran attacks and they failed" to "the harness makes this outcome unrepresentable," which holds independent of model behaviour. That is a useful corrective to the reflex of grading security papers by proof difficulty — the right question is what the proof makes independent of, not how hard it was.

Watch for inverted emphasis. The ordering-variance measurement — 43 % and 33 % standard deviations, bimodal outcomes at fixed N — is a transferable result about how this whole class of experiment must be powered, and it sits unnamed in a mid-section paragraph. The elementary theorem gets a titled appendix. When reading, it is worth asking which finding would survive longest if the paper's specific attack were patched tomorrow; here it is the variance result, and the paper does not treat it that way.

Sources: Paper to read — user-curated self-email, 2026-07-27 · arXiv:2605.01970